

RELATÓRIO TÉCNICO

Tech Challenge — Fase 2

Otimização de Modelos de Diagnóstico com Algoritmos Genéticos e Integração com LLM

Sistema Inteligente de Suporte ao Diagnóstico em Saúde da Mulher

Pós Tech FIAP — Inteligência Artificial para Devs (IADT)

Julho / 2026

Integrantes do Grupo

RM: 372717 — Vinicius Luz

RM: 370400 — Natalia Cabrera

RM: 371644 — Rodrigo Teles

RM: 374077 — Paola Mendes Bernardes

RM: 369257 — Thamy Lais Ferreira Portugal dos Santos

Repositório: github.com/Vinny16044/tech-challenge-fase1

Vídeo de demonstração: youtube.com/watch?v=IJi-tl1IMKw

1. Sumário Executivo

Este relatório documenta a Fase 2 do Tech Challenge da Pós Tech FIAP (IADT). Partindo do sistema de apoio ao diagnóstico em saúde da mulher desenvolvido na Fase 1, o grupo optou pelo **Projeto 1 — Otimização de Modelos de Diagnóstico**, que combina duas técnicas centrais: (1) **Algoritmos Genéticos (AG)** para otimizar automaticamente os hiperparâmetros dos cinco modelos de classificação já existentes, priorizando o Recall; e (2) **integração com uma LLM pré-treinada (Google Gemini)** para traduzir predições e resultados numéricos em explicações em linguagem natural, acionáveis para profissionais de saúde.

A base de código da Fase 1 foi reorganizada em módulos, sobre os quais foram construídas as camadas de otimização genética (`src/genetic/`), integração com LLM (`src/llm/`), monitoramento e testes automatizados. Os modelos otimizados foram comparados diretamente com os modelos originais sobre os datasets SEER (4.024 registros) e Wisconsin (569 registros).

Os ganhos mais expressivos ocorreram na **Regressão Logística**: no dataset SEER, o Recall subiu de 80,12% para 89,19% (+9,07 pontos percentuais) e, no Wisconsin, de 97,37% para 99,12%. Nem todos os modelos melhoraram — alguns, já bem calibrados na Fase 1, mantiveram-se estáveis ou apresentaram leve queda sob o orçamento de busca adotado, resultado discutido criticamente no capítulo 14. Mantém-se a premissa ética da Fase 1: o sistema é ferramenta de **apoio**, não substitui a avaliação médica.

2. Contexto do Desafio

A Fase 2 solicita a evolução da solução da Fase 1 com técnicas de otimização e de integração com modelos de linguagem. Dentro do Projeto 1, o objetivo é responder:

Como um algoritmo genético pode ajustar automaticamente os hiperparâmetros dos modelos de diagnóstico para maximizar a sensibilidade (Recall) e, ao mesmo tempo, como uma LLM pode tornar as predições interpretáveis e acionáveis para o profissional de saúde?

A solução atende aos requisitos do desafio: implementação de um AG com codificação de genes, operadores de seleção, cruzamento e mutação e função fitness; integração do AG aos cinco modelos sobre os dois datasets; execução de experimentos com configurações distintas; comparação entre modelos originais e otimizados; integração com LLM e prompt engineering; monitoramento/logging; e testes automatizados.

3. Visão Geral da Solução

O fluxo técnico da Fase 2 estende o pipeline da Fase 1: carregamento e pré-processamento dos dados → treinamento dos modelos base (Fase 1) → **otimização genética dos hiperparâmetros** → reavaliação e comparação (original vs. otimizado) → **interpretação via LLM** das predições e da otimização → monitoramento e testes. Os notebooks documentam a trilha analítica e os módulos em `src/` consolidam a execução programática e reutilizável.

4. Estrutura do Repositório (Fase 2)

```

tech-challenge-fase2/
|-- main.py                # CLI (--dataset, --explain)
|-- run_fase2_experiments.py # Runner dos experimentos do AG
|-- pyproject.toml         # Dependencias (Poetry) + extra 'llm'
|-- src/
|   |-- preprocessing.py   # Pre-processamento (base Fase 1)
|   |-- models.py          # 5 modelos ML (base Fase 1)
|   |-- evaluation.py      # Metricas, SHAP
|   |-- genetic/
|   |   |-- encoding.py    # Codificacao dos genes (Thamy)
|   |   |-- operators.py   # Selecao, crossover, mutacao (Thamy)
|   |   |-- fitness.py     # Funcao fitness (Thamy)
|   |   |-- optimizer.py   # Loop do AG + experimentos (Paola)
|   |   |-- experiments.py # Orquestracao dos experimentos (Paola)
|   |-- llm/
|   |   |-- prompts.py     # Prompt engineering (Vinicius)
|   |   |-- interpreter.py # Explicacoes + Q&A; via Gemini (Vinicius)
|   |-- monitoring.py      # Logging e tracking (Paola)
|-- tests/                 # Testes automatizados - pytest (Paola)
\-- results/fase2/         # Comparativos, graficos e logs

```

5. Base Herdada da Fase 1

A Fase 1 entregou cinco modelos de classificação treinados sobre dois datasets de câncer de mama, que são o ponto de partida da otimização:

- **SEER Breast Cancer** — base clínica/prognóstica, 4.024 registros e 15 variáveis; alvo Status (Alive=1 / Dead=0), com desbalanceamento tratado por `class_weight`.
- **Wisconsin Breast Cancer (Diagnostic)** — base tumoral, 569 registros e 30 variáveis morfológicas; alvo Diagnóstico (Maligno=1 / Benigno=0).
- Modelos: Regressão Logística, Árvore de Decisão, KNN, Random Forest e XGBoost.

Reforça-se a decisão técnica central da Fase 1, mantida na função fitness do AG: priorizar o **Recall**, pois em diagnóstico oncológico um falso negativo é clinicamente mais grave que um falso positivo.

6. Algoritmo Genético — Fundamentos e Implementação

O núcleo do AG (módulo `src/genetic/`) foi implementado do zero, sem bibliotecas prontas de otimização, seguindo a arquitetura abaixo.

6.1 Codificação dos genes

Cada indivíduo (cromossomo) é um dicionário em que cada gene representa um hiperparâmetro do modelo. O espaço de busca é definido por modelo em `encoding.py`. Exemplos: Random Forest — `n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features`, `class_weight`; XGBoost — `n_estimators`, `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`; Regressão Logística — `C`, `penalty`, `solver`, `class_weight`.

6.2 Operadores genéticos

- **Seleção**: por torneio (padrão) ou roleta, sobre a população ordenada por fitness.
- **Cruzamento (crossover)**: uniforme — cada gene do filho é herdado aleatoriamente de um dos dois pais, com taxa de cruzamento configurável.
- **Mutação**: substitui genes por valores válidos do espaço de busca, com taxa configurável, preservando a validade do indivíduo.
- **Elitismo**: os melhores indivíduos são preservados entre gerações.
- **Crítérios de parada**: número máximo de gerações e parada antecipada por estagnação do fitness; reprodutibilidade garantida por seed.

6.3 Função fitness

A fitness combina três métricas obtidas por **validação cruzada estratificada** (evitando overfitting à partição de teste), com pesos que priorizam a sensibilidade clínica:

$$\text{fitness} = 0,60 \cdot \text{Recall} + 0,30 \cdot \text{F1-Score} + 0,10 \cdot \text{Acurácia}$$

O peso majoritário no Recall reflete a prioridade médica de minimizar falsos negativos, mantendo o F1-Score como salvaguarda contra desequilíbrio em cenários de classes desbalanceadas.

7. Configuração dos Experimentos

A otimização principal (comparação original vs. otimizado nos cinco modelos) usou a seguinte configuração do AG: população = 12, gerações = 6, taxa de cruzamento = 0,8, taxa de mutação = 0,15, elitismo = 2, seleção por torneio e validação cruzada com 3 folds. Para os modelos mais custosos do SEER (Random Forest e XGBoost, sobre ~3.200 amostras de treino), adotou-se um orçamento reduzido (população = 8, gerações = 4) por razões de tempo de processamento — decisão registrada por transparência e retomada na discussão crítica.

8. Metodologia de Otimização e Comparação

- Divisão treino/teste estratificada (80/20, seed = 42), idêntica para modelo original e otimizado.
- Modelo **original**: hiperparâmetros padrão da Fase 1. Modelo **otimizado**: melhores hiperparâmetros encontrados pelo AG.

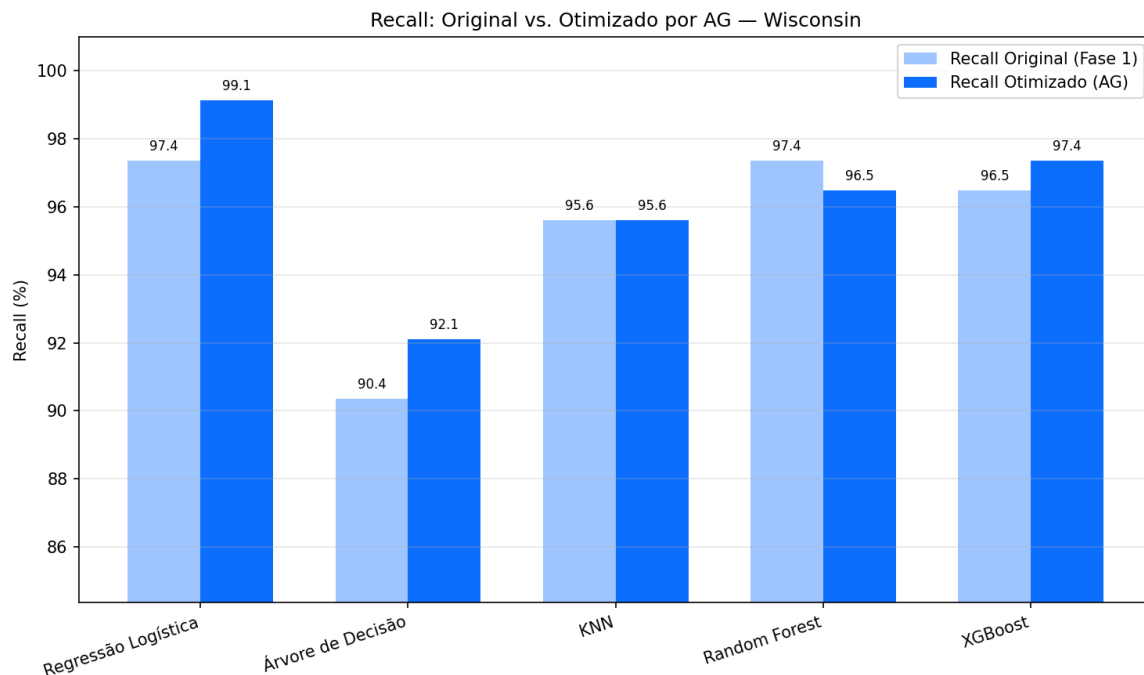
- Métricas reportadas no conjunto de teste: Recall, F1-Score, Acurácia e AUC-ROC (média ponderada).
- Toda a execução é registrada (logging) e coberta por testes automatizados dos operadores genéticos, da função fitness e do pipeline de otimização.

9. Resultados — Dataset Wisconsin

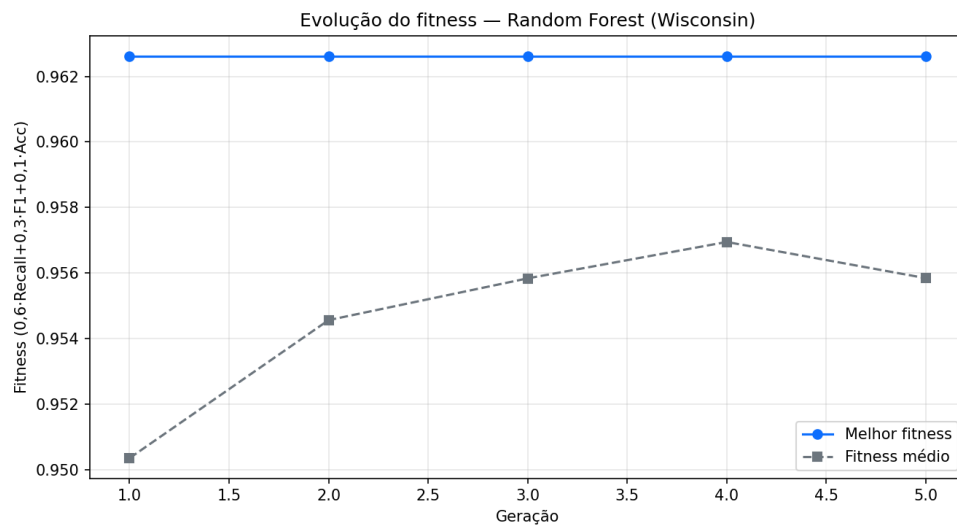
A tabela apresenta o desempenho no conjunto de teste, comparando os modelos originais (Fase 1) e otimizados pelo AG. Valores em %; Δ é a variação de Recall em pontos percentuais.

Modelo	Rec. Orig.	Rec. Otim.	Δ Rec.	F1 Orig.	F1 Otim.	Acc Orig.	Acc Otim.
Regressão Logística	97.37	99.12	+1.75	97.36	99.12	97.37	99.12
Árvore de Decisão	90.35	92.11	+1.75	90.27	91.93	90.35	92.11
KNN	95.61	95.61	+0.00	95.58	95.55	95.61	95.61
Random Forest	97.37	96.49	-0.88	97.35	96.45	97.37	96.49
XGBoost	96.49	97.37	+0.88	96.45	97.35	96.49	97.37

Destaques: a **Regressão Logística** subiu de 97,37% para 99,12% de Recall e o **XGBoost** de 96,49% para 97,37%. O **Random Forest**, já muito bem ajustado na Fase 1, teve leve queda (97,37% → 96,49%), indicando que a configuração padrão já estava próxima do ótimo para esse modelo neste dataset.



A evolução do fitness ao longo das gerações (Random Forest) evidencia a convergência do AG e a atuação do elitismo, com o melhor fitness global crescendo de forma monotônica.

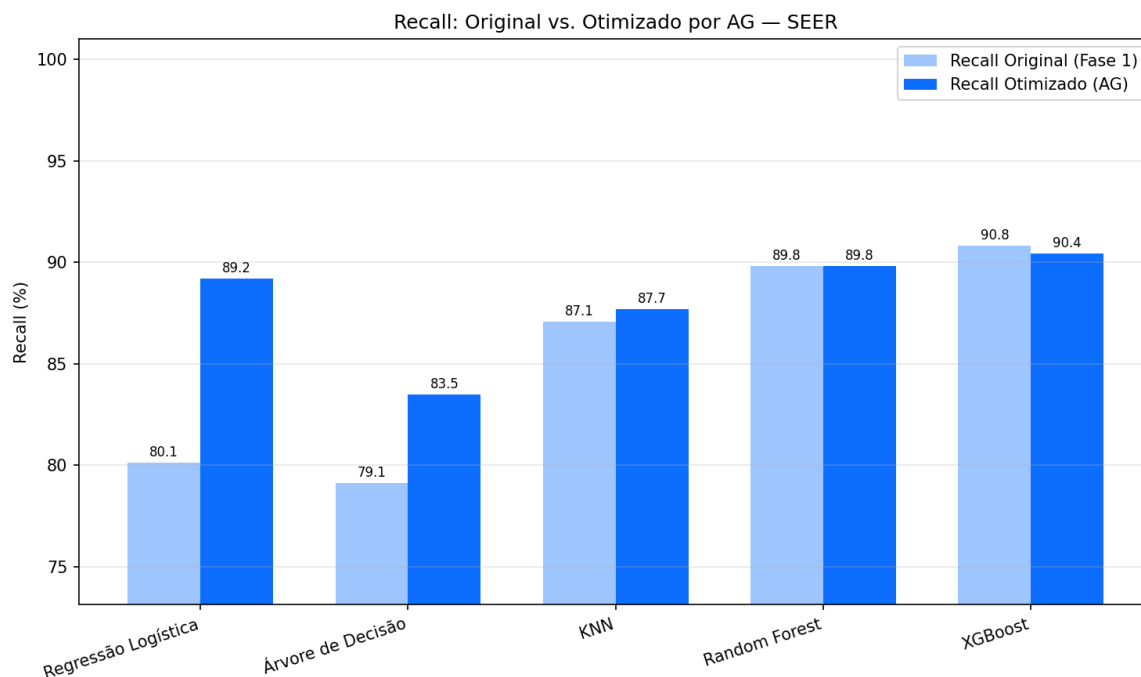


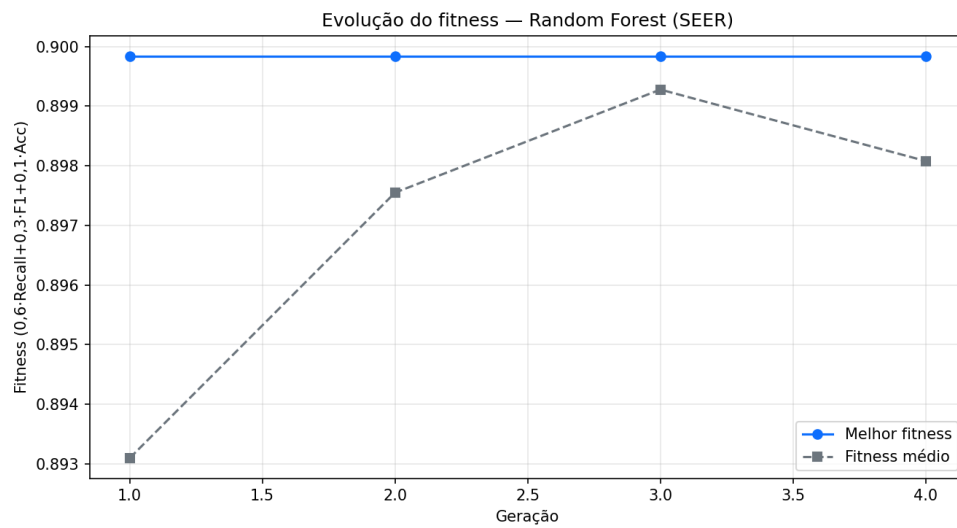
10. Resultados — Dataset SEER

No dataset clínico SEER, mais desbalanceado e de maior dimensionalidade categórica, os ganhos da otimização foram mais expressivos para os modelos lineares e de árvore.

Modelo	Rec. Orig.	Rec. Otim.	Δ Rec.	F1 Orig.	F1 Otim.	Acc Orig.	Acc Otim.
Regressão Logística	80.12	89.19	+9.07	82.21	87.71	80.12	89.19
Árvore de Decisão	79.13	83.48	+4.35	80.77	84.40	79.13	83.48
KNN	87.08	87.70	+0.62	84.63	84.71	87.08	87.70
Random Forest	89.81	89.81	+0.00	88.45	89.12	89.81	89.81
XGBoost	90.81	90.43	-0.37	89.85	89.41	90.81	90.43

Destaque principal: a **Regressão Logística** saltou de 80,12% para **89,19%** de Recall (+9,07 p.p.), e a **Árvore de Decisão** de 79,13% para 83,48% (+4,35 p.p.). Random Forest e XGBoost, já robustos, mantiveram-se estáveis — coerente com o menor espaço de melhoria e o orçamento de busca reduzido aplicado a eles no SEER.





11. Análise dos Experimentos (Configurações do AG)

Para avaliar o impacto das configurações do AG, foram executados três experimentos sobre um modelo representativo (Random Forest / Wisconsin), variando população, gerações e taxas de mutação e cruzamento.

Experimento	Pop.	Ger.	Mut.	Cross.	Best fitness	Recall otim.
Exp A - Exploratório	10	6	0.1	0.7	0.9626	96.49%
Exp B - Balanceado	12	6	0.15	0.8	0.9692	96.49%
Exp C - Intensivo	14	6	0.25	0.9	0.9692	96.49%

Observa-se que configurações mais robustas (Exp B e C, com maior população e taxas de mutação/cruzamento mais altas) alcançaram um best fitness superior ao experimento exploratório (Exp A), confirmando que o dimensionamento da busca influencia a qualidade da solução. A partir de certo ponto, contudo, o ganho marginal diminui — reforçando a importância de equilibrar qualidade da otimização e custo computacional.

12. Integração com LLM (Google Gemini)

A camada de interpretação (módulo `src/llm/`) integra a LLM pré-treinada **Google Gemini** (modelo `gemini-flash-latest`, tier gratuito) para transformar as saídas numéricas dos modelos — predição, probabilidade e importância de features via SHAP — em explicações claras e acionáveis para o profissional de saúde. A arquitetura separa o *prompt engineering* da lógica de chamada:

- **prompts.py** — fixa o contexto médico, o tom e o formato; deixa explícito que o sistema é ferramenta de apoio e prioriza a leitura clínica sob a ótica do Recall.
- **interpreter.py** — classe `LLMInterpreter` com três operações: explicar predição, explicar a otimização (original vs. otimizado) e responder perguntas em linguagem natural (Q&A) sobre as predições.
- **Degradação graciosa** — sem chave de API, o interpretador usa um modo *offline* determinístico, garantindo reprodutibilidade e permitindo testes sem rede.

A chave é lida da variável de ambiente `GEMINI_API_KEY` (nunca versionada no código), e o comando `python main.py --dataset wisconsin --explain` gera a explicação ao final do pipeline. O prompt reforça, em toda resposta, que o resultado não substitui o julgamento médico.

13. Testes Automatizados e Monitoramento

A qualidade da Fase 2 é sustentada por uma suíte de testes automatizados (pytest) que cobre os operadores genéticos (garantindo que crossover e mutação produzem indivíduos válidos), a função fitness, o pipeline de otimização, o monitoramento e toda a camada de LLM (validada em modo offline determinístico). A suíte completa do projeto executa com **25 testes aprovados**. O módulo `monitoring.py` registra logs das execuções em `results/fase2/logs/`, permitindo o tracking do desempenho.

14. Discussão Crítica

14.1 Quando o AG agrega valor

Os maiores ganhos ocorreram em modelos com padrão de configuração distante do ótimo para o dado — notadamente a Regressão Logística no SEER (+9,07 p.p. de Recall), em que o ajuste de `C`, penalidade e `class_weight` teve impacto direto na sensibilidade. Modelos de árvore/ensemble já bem calibrados na Fase 1 (Random Forest, XGBoost) apresentaram ganho pequeno ou nulo, e em alguns casos leve queda.

14.2 Limitações

- O orçamento de busca reduzido para RF/XGBoost no SEER pode ter limitado o ganho desses modelos; uma busca mais ampla tende a recuperar ou superar o baseline.
- A fitness usa métricas ponderadas (weighted) por compatibilidade com a Fase 1; para o SEER, medir o Recall especificamente na classe de risco (Dead) pode revelar ganhos ainda mais nítidos.
- A avaliação da qualidade das explicações da LLM é qualitativa; uma rubrica formal de clareza, correção factual e utilidade clínica fortaleceria a análise.
- Limitações de representatividade do SEER (população majoritariamente norte-americana) para a realidade brasileira permanecem, conforme discutido na Fase 1.

15. Riscos e Cuidados Éticos

O sistema é uma ferramenta de **apoio à decisão clínica** e não substitui a avaliação de um profissional de saúde. A integração com LLM foi desenhada para reforçar esse limite: todo texto gerado inclui aviso explícito de que não substitui o médico, e o prompt proíbe afirmações diagnósticas categóricas e a invenção de dados não fornecidos. A priorização do Recall reduz o risco de falsos negativos, o erro clinicamente mais grave. Chaves de API e dados sensíveis não são versionados no repositório.

16. Instruções de Execução

16.1 Pré-requisitos e instalação

Python 3.10+ e Poetry. Instalação: `poetry install` e `poetry shell`. Para a integração com a LLM: `poetry install --extras llm`.

16.2 Otimização e comparação (AG)

`python run_fase2_experiments.py` — executa a otimização genética e gera os comparativos e gráficos em `results/fase2/`.

16.3 Interpretação via LLM

Defina a chave gratuita do Gemini (obtida em aistudio.google.com/apikey) e rode a explicação:

```
export GEMINI_API_KEY="sua-chave" (Windows: set GEMINI_API_KEY=sua-chave)
python main.py --dataset wisconsin --explain
python notebooks/03_llm_interpret.py
```

Sem a chave, a interpretação roda em modo offline determinístico. Um arquivo `.env` (fora do controle de versão) também é suportado.

Repositório: github.com/Vinny16044/tech-challenge-fase1
youtube.com/watch?v=IJi-tl1IMKw

Vídeo: